

CRISP: Circuit-pRivate Single-Image Steganography with Permutations

Shahzad Ahmad^[0000–0002–9654–869X] and Stefan Rass^[0000–0003–2821–2489]

LIT Secure and Correct Systems Lab, Johannes Kepler University Linz,
Altenbergerstrasse 69, 4040 Linz, Austria
`shahzad.ahmad@jku.at`, `stefan.rass@jku.at`

Abstract. We introduce CRISP (Circuit-pRivate Single-Image Steganography with Permutations), a provably secure homomorphic steganography scheme designed to address key limitations in existing approaches to private computation. Current methods often suffer from excessive image overhead and a complete lack of circuit privacy. Specifically, many techniques require a separate cover image for each wire in a circuit, leading to a large number of images for even moderately sized circuits. This also inadvertently reveals the underlying logical gate structure to an honest-but-curious cloud provider, as wire roles are often fixed to specific images. CRISP resolves these challenges by embedding all three Fredkin gate inputs directly into the RGB channels of a *single* cover image at a secret pixel position (*row, col*). Similarly, all three outputs are written to a single output image. To ensure robust circuit privacy, CRISP employs two independently sampled permutations: π_{in} assigns input channel roles, and π_{out} reassigns output channel roles. These permutations work in tandem to prevent an adversary from deducing the circuit’s structure. Without π_{out} varying per gate, the Fredkin gate’s pass-through of the control bit could enable a deterministic channel-matching attack, fully exposing the logical gate structure to the cloud provider. By resampling π_{in} and π_{out} independently for each gate, CRISP effectively prevents cross-gate correlation attacks.

Our security analysis demonstrates that an adversary’s advantage in locating the secret pixel and inferring the embedded bits decays as $\Theta(1/(h \times w))$, where $h \times w$ is the image resolution. This bound is cryptographically negligible for any practical image size. CRISP significantly reduces image overhead: per gate, it uses just two images (one input cover plus one independent output cover). This represents a substantial reduction compared to other methods that might require multiple images per wire. For an obfuscated benchmark circuit, CRISP further demonstrates a notable reduction in total image count. Crucially, CRISP delivers circuit privacy, a qualitatively new and essential security guarantee.

The correctness, security, and efficiency of CRISP are formally proved and experimentally validated.

Keywords: Homomorphic steganography · Circuit privacy · Fredkin gate · privacy-preserving computation · Cloud computing · Reversible logic

1 Introduction

Cloud computing has transformed how organisations store and process sensitive data. Outsourcing computation to third-party cloud providers offers scalability and cost efficiency, but introduces a significant privacy challenge: enabling an untrusted server to compute on data without revealing any information about it.

Existing solutions address this from different angles. On the one hand, Fully Homomorphic Encryption (FHE) [18] enables arbitrary computation over encrypted data, but its computational overhead remains prohibitive for large-scale deployment. On the other hand, Trusted Execution Environments (TEEs) [12] offer hardware-level isolation but depend on specific CPU features, require complex attestation protocols, and remain vulnerable to side-channel attacks [6]. Meanwhile, traditional steganography hides the existence of a message but provides no mechanism for computation on hidden data.

Provably Secure Homomorphic Steganography (ProSt) [2] introduces a new approach: hiding secret bits in cover images via LSB steganography and enabling homomorphic computation on them using Fredkin gates [16]. An honest-but-curious cloud provider processes each pixel without knowing which carries the secret. ProSt offers provable security; the adversary’s advantage decreases as candidate pixel-channel positions increase.

ProSt faces two key limitations. First, each circuit wire needs a dedicated cover image. A circuit with 61 wires requires 61 images, creating significant communication and storage overhead. Second, ProSt lacks circuit privacy: since wire roles are tied to specific images, the cloud can always identify which image holds the control bit at each gate, exposing the computation’s logical structure. *Our Contributions.* This paper introduces CRISP, which resolves both limitations through:

1. **Single-image Fredkin embedding.** All three Fredkin gate inputs are embedded into the three RGB channels of a single input cover image at a secret pixel position (row, col) . All three outputs are written into a single, independently sampled output cover image at the same position. Each gate consumes exactly 2 images, one input cover and one output cover, compared to ProSt’s 6 per gate (three input wire images and three output wire images). For an n -gate circuit, CRISP requires $2n$ images in total, versus ProSt’s L images for L circuit wires, giving a $3\times$ per-gate reduction and a $L/(2n)$ circuit-level reduction.
2. **Dual independent permutations.** A secret input permutation $\pi_{in} \in S_3$ assigns the three logical roles (control, x , y) to physical channels $\{R, G, B\}$ before embedding. A separate, independently sampled output permutation $\pi_{out} \in S_3$ reassigns channel roles when the cloud writes Fredkin outputs. Both permutations are part of the public circuit specification: Carol needs π_{in} to read the correct logical inputs and π_{out} to write the correct outputs. The circuit privacy guarantees arise from π_{out} varying independently per gate.

3. **Formal security analysis with Bayesian advantage decay.** We prove that the adversary’s advantage in inferring the embedded secret is $\Theta(1/n)$ with $n = h \times w$. The proof is self-contained and relies on a direct Bayesian calculation that exploits the exchangeability of the image LSB vector.
4. **Necessity of dual permutation for circuit privacy.** We formally show that a single permutation is insufficient: the Fredkin gate’s pass-through of the control bit enables a channel-matching attack that deterministically identifies the control channel without π_{out} .

The remainder of this paper is organised as follows. Section 2 reviews related work. Section 3 provides background on LSB steganography, Fredkin gates, and ProSt. Section 4 presents the CRISP formal scheme. Section 5 provides the security analysis. Section 6 describes the implementation. Section 7 presents experimental results. Section 8 concludes.

2 Related Work

Steganography. LSB substitution [9] replaces the least significant bit of pixel values with secret bits, achieving high capacity with negligible perceptual distortion. More sophisticated spatial-domain techniques including pixel-value differencing [29] and adaptive methods [26] improve resistance to steganalysis. Transform-domain approaches such as DCT-based JPEG steganography and DWT-based schemes hide data in frequency coefficients for stronger statistical imperceptibility. Deep learning methods [3,32] jointly train encoder-decoder networks for end-to-end imperceptibility and robustness. All of these methods treat steganography purely as a hiding mechanism: none support computation on hidden data without first extracting the secret.

LSB substitution replaces the least significant bit of pixel values with secret bits, achieving high capacity with negligible perceptual distortion. More sophisticated spatial-domain techniques including pixel-value differencing and adaptive methods improve resistance to steganalysis. Transform-domain approaches such as DCT-based JPEG steganography and DWT-based scheme hide data in frequency coefficients for stronger statistical imperceptibility. Deep learning methods jointly train encoder-decoder networks for end-to-end imperceptibility and robustness. All of these methods treat steganography purely as hiding mechanism: none support computation on hidden data without first extracting the secret.

Homomorphic Encryption. Gentry’s seminal work [18] established FHE, enabling the arbitrary evaluation of circuits over ciphertexts. Subsequent schemes, including BGV [5], BFV [15], CKKS [10], and TFHE [11], significantly improved practical efficiency. Despite these advances, FHE incurs substantial computational overhead due to bootstrapping, complex key management, and ciphertext expansion. CRISP avoids encryption entirely, instead exploiting LSB statistical properties to achieve privacy during computation.

Trusted Execution Environments. Intel SGX [12], ARM TrustZone, and AMD SEV provide hardware-enforced isolation for sensitive computations. However, they depend on specific hardware features, require complex remote attestation, and remain vulnerable to side channel attacks [6,25]. CRISP requires no specialised hardware and is immune to these concerns since the cloud provider computes openly on all pixels.

Secure Multi-Party Computation. Yao’s garbled circuits [30] and the GMW protocol [20] established MPC foundations, with subsequent work including SPDZ [13], improving efficiency. MPC protocols generally require multiple rounds of interaction, which is impractical for asynchronous cloud outsourcing. CRISP requires zero interaction between Alice and the cloud during computation.

Garbled Circuits and Circuit Privacy. Circuit privacy in garbled circuits [30,4] ensures that a garbled circuit reveals nothing about the circuit topology beyond its input-output behaviour. CRISP introduces an analogous notion in the steganographic setting: the cloud provider cannot determine which channel plays which logical role at any gate. This is achieved without encryption overhead.

ProSt. ProSt introduced [2] homomorphic steganography, enabling provably secure computation on LSB-hidden data using Fredkin gates. ProSt’s per-wire per-image architecture and lack of circuit privacy are the direct motivations for CRISP.

3 Background

3.1 LSB Steganography

An image I is a three-dimensional tensor $I \in \mathbb{Z}_{256}^{h \times w \times 3}$, where h, w are height and width in pixels and the third dimension indexes channels $\{0, 1, 2\} = \{R, G, B\}$.

Definition 1 (LSB Embedding and Extraction). For image I , secret bit $b \in \{0, 1\}$, and position (row, col, k) :

$$\text{Emb}(I, b, row, col, k) : I[row, col, k] \leftarrow (I[row, col, k] \& \neg 1) \mid b \quad (1)$$

$$\text{Ext}(I, row, col, k) : b \leftarrow I[row, col, k] \& 1 \quad (2)$$

The modification changes pixel intensity by at most 1, which is imperceptible. Security relies not on imperceptibility alone but on positional secrecy: without knowing (row, col) , an adversary cannot identify which of the $h \times w$ pixels carries the secret.

3.2 The Fredkin Gate

The Fredkin gate $F : \{0, 1\}^3 \rightarrow \{0, 1\}^3$ is a universal reversible logic gate defined as:

$$F(c, x, y) = \begin{cases} (c, x, y) & \text{if } c = 0 \\ (c, y, x) & \text{if } c = 1 \end{cases} \quad (3)$$

where c is the control bit and x, y are data bits. Two properties are central to CRISP.

Lemma 1 (Universality [16]). *The Fredkin gate can simulate AND, OR, and NOT:*

$$\text{AND}(a, b) = F(a, b, 0)_y \quad \text{NOT}(x) = F(x, 1, 0)_x \quad \text{OR}(a, b) = F(a, 1, b)_y$$

where subscripts denote output wire selection. Any Boolean function can be expressed as a network of Fredkin gates.

Lemma 2 (Hamming Weight Invariance [2]). *For any $(c, x, y) \in \{0, 1\}^3$, the Hamming weight satisfies $c + x + y = c_{\text{out}} + x_{\text{out}} + y_{\text{out}}$, where $(c_{\text{out}}, x_{\text{out}}, y_{\text{out}}) = F(c, x, y)$.*

3.3 ProSt Summary

ProSt [2] assigns one cover image per circuit wire. For each gate G_i with inputs (I_c, I_1, I_2) and outputs (O_c, O_1, O_2) , the cloud applies:

$$(\text{lsb}(O_c[r, c, k]), \text{lsb}(O_1[r, c, k]), \text{lsb}(O_2[r, c, k])) \leftarrow F(\text{lsb}(I_c[r, c, k]), \text{lsb}(I_1[r, c, k]), \text{lsb}(I_2[r, c, k]))$$

at every pixel (r, c) and channel k . Alice embeds her secret into each image at an independently sampled position (r_i, c_i, k_i) from $h \times w \times 3$ possibilities.

Limitations. (i) ProSt allocates one cover image per circuit wire. For each Fredkin gate, the cloud accesses 3 input wire images and writes to 3 output wire images, for a total of 6 images per gate. Input wire images are shared across gates (e.g., ancillary wires such as const_0 and const_1 appear as inputs to many gates), so the total distinct image count equals the wire count L , not $6n$. For the 17-gate obfuscated benchmark, $L = 61$. (ii) Wire roles are fixed to specific images, so the cloud always knows which image carries the control bit at each gate; there is no circuit privacy. CRISP addresses both.

4 CRISP: Formal Scheme Definition

4.1 Overview

CRISP maps the three inputs (c, x, y) of a Fredkin gate to the three RGB channels of a single input image I_{in} at a secret pixel position (row, col) , and maps the three outputs to the three channels of a single output image I_{out} at the same pixel position. Two independently sampled permutations control channel-to-role assignment: $\pi_{\text{in}} \in S_3$ assigns logical roles to channels before embedding, and $\pi_{\text{out}} \in S_3$ assigns output roles after Fredkin computation. Both permutations are part of the public circuit specification transmitted to Carol; Carol requires $\pi_{\text{in}} \in S_3$ to read the correct logical inputs and $\pi_{\text{out}} \in S_3$ to write outputs to the correct channels. The secret known only to Alice and Bob is the pixel position (row, col) .

4.2 System Parameters

Let images have fixed dimensions $h \times w \times 3$. A Boolean circuit C is a DAG with:

- Input wires W_{in} , intermediate wires W_{mid} , output wires W_{out}
- n Fredkin gates $\mathcal{G} = \{G_1, \dots, G_n\}$ in valid topological order
- Each gate G_i has input triple (c_i, x_i, y_i) and output triple $(c_i^{\text{out}}, x_i^{\text{out}}, y_i^{\text{out}})$

4.3 Secret Key

The secret key shared between Alice and Bob is:

$$K = \{(row_i, col_i)\}_{i=1}^n \quad (4)$$

where $(row_i, col_i) \in \{0, \dots, h-1\} \times \{0, \dots, w-1\}$ is the secret pixel position for gate G_i . The permutations $\pi_{\text{in}}^{(i)}, \pi_{\text{out}}^{(i)} \in S_3$ are independently sampled per gate and included in the public circuit specification. K is communicated from Alice to Bob over a private channel before any computation.

4.4 Algorithms

Algorithm 1: LocGen(h, w)

```

1:  $row \xleftarrow{\$} \{0, \dots, h-1\}$ 
2:  $col \xleftarrow{\$} \{0, \dots, w-1\}$ 
3: return  $(row, col)$ 

```

Algorithm 2: PermGen()

```

1:  $\pi_{\text{in}} \xleftarrow{\$} S_3$  {Uniform over all 6 permutations of  $\{0, 1, 2\}$ }
2:  $\pi_{\text{out}} \xleftarrow{\$} S_3$  {Independent of  $\pi_{\text{in}}$ }
3: return  $(\pi_{\text{in}}, \pi_{\text{out}})$ 

```

Algorithm 3: Emb($I, c_{\text{bit}}, x_{\text{bit}}, y_{\text{bit}}, row, col, \pi_{\text{in}} \sim$ executed by Alice)

```

1:  $I' \leftarrow I$ 
2:  $bits[0] \leftarrow c_{\text{bit}}; bits[1] \leftarrow x_{\text{bit}}; bits[2] \leftarrow y_{\text{bit}}$ 
3: for  $t \in \{0, 1, 2\}$  do
4:    $I'[row, col, \pi_{\text{in}}(t)] \leftarrow (I[row, col, \pi_{\text{in}}(t)] \& \text{0xFE}) \mid bits[t]$ 
5: return  $I'$ 

```

Algorithm 4: Comp($I_{\text{in}}, I_{\text{out}}, \pi_{\text{in}}, \pi_{\text{out}} \sim$ executed by Carol)

```

1:  $result \leftarrow I_{\text{out}}.copy()$ 
2: for each pixel  $(row', col') \in \{0, \dots, h-1\} \times \{0, \dots, w-1\}$  do

```

```

3:   $c_{\text{bit}} \leftarrow I_{\text{in}}[\text{row}', \text{col}', \pi_{\text{in}}(0)] \& 1$ 
4:   $x_{\text{bit}} \leftarrow I_{\text{in}}[\text{row}', \text{col}', \pi_{\text{in}}(1)] \& 1$ 
5:   $y_{\text{bit}} \leftarrow I_{\text{in}}[\text{row}', \text{col}', \pi_{\text{in}}(2)] \& 1$ 
6:   $(c_{\text{out}}, x_{\text{out}}, y_{\text{out}}) \leftarrow F(c_{\text{bit}}, x_{\text{bit}}, y_{\text{bit}})$ 
7:   $\text{out}[0] \leftarrow c_{\text{out}}; \text{out}[1] \leftarrow x_{\text{out}}; \text{out}[2] \leftarrow y_{\text{out}}$ 
8:  for  $t \in \{0, 1, 2\}$  do
9:     $\text{result}[\text{row}', \text{col}', \pi_{\text{out}}(t)] \leftarrow (\text{result}[\text{row}', \text{col}', \pi_{\text{out}}(t)] \& 0\text{xFE}) \mid \text{out}[t]$ 
10: return  $\text{result}$ 

```

Algorithm 5: $\text{Ext}(I_{\text{out}}, \text{row}, \text{col}, \pi_{\text{out}} \sim)$ executed by Bob

```

1: for  $t \in \{0, 1, 2\}$  do
2:    $\text{bits}[t] \leftarrow I_{\text{out}}[\text{row}, \text{col}, \pi_{\text{out}}(t)] \& 1$ 
3: return  $(\text{bits}[0], \text{bits}[1], \text{bits}[2]) \{(c_{\text{out}}, x_{\text{out}}, y_{\text{out}})\}$ 

```

4.5 Remark on Carol's Knowledge of π_{in} and π_{out}

Both π_{in} and π_{out} are included in the circuit specification transmitted to Carol. Under the honest-but-curious model, Carol follows the protocol exactly. She requires π_{in} to read the correct logical roles from the input channels (Algorithm 4, lines 2–4), and π_{out} to write each output bit to the correct channels (lines 8–10). Without the secret pixel position (row, col) , Carol cannot identify which pixel carries the actual secret, even with full knowledge of both permutations. The sole secret in CRISP is the position (row, col) , which is never transmitted and is known only to Alice and Bob.

The circuit privacy guarantees do not stem from hiding π_{in} ; rather, they arise from the fact that π_{out} is resampled independently per gate, which breaks the per-channel LSB correlation attack described in Theorem 2 (see also Remark 1 for the precise scope of this guarantee).

4.6 Multi-Gate Circuit Evaluation

For a circuit with n gates in topological order:

1. *Key Generation*: Alice runs LocGen for each gate G_i , producing $K = \{(\text{row}_i, \text{col}_i)\}_{i=1}^n$. She also runs PermGen per gate to generate $(\pi_{\text{in}}^{(i)}, \pi_{\text{out}}^{(i)})$, which are included in the public circuit specification.
2. *Embedding*: Alice embeds the circuit input bits into $I_{\text{in}}^{(1)}$ at position $(\text{row}_1, \text{col}_1)$ using $\pi_{\text{in}}^{(1)}$. Subsequent gate inputs propagate automatically through Carol's pixel-wise computation; no re-embedding is required.
3. *Computation*: Carol evaluates gates in topological order. For gate G_i , she applies $\text{Comp}(I_{\text{in}}^{(i)}, I_{\text{out}}^{(i)}, \pi_{\text{in}}^{(i)}, \pi_{\text{out}}^{(i)})$. The output image $I_{\text{out}}^{(i)}$ serves as $I_{\text{in}}^{(i+1)}$ for linear chains. For general DAGs, wire tracking routes outputs to the correct subsequent gate inputs.
4. *Extraction*. Bob applies Ext to $I_{\text{out}}^{(n)}$ at position $(\text{row}_n, \text{col}_n)$ with $\pi_{\text{out}}^{(n)}$ to recover the final output bits.

4.7 Image Count

ProSt allocates one cover image per circuit wire and maps each wire name to its own image throughout computation. For a Fredkin gate with inputs (c -wire, x -wire, y -wire) and outputs (c_{out} -wire, x_{out} -wire, y_{out} -wire), ProSt reads three input wire images and writes three output wire images, for a total of 6 images per gate. Because ancillary wires such as const_0 and const_1 appear as inputs to many gates, these input images are shared across the circuit. The total distinct image count, therefore, equals the number of distinct wire L , not $6n$.

CRISP replaces the wire-centric model with a gate-centric one. Each gate G_i is assigned one fresh input cover $I_{\text{in}}^{(i)}$ and one independent output cover $I_{\text{out}}^{(i)}$: 2 images per gate. No image is shared between gates. Wire values propagate between gates as scalars in the signal dictionary, not as images.

Proposition 1 (Image Count). *A circuit of n Fredkin gates requires exactly $2n$ cover images under CRISP (n input covers and n output covers, all independently sampled) and L images under ProSt (L = number of distinct circuit wires).*

This yields two distinct and complementary reduction figures:

- **Per-gate:** $6/2 = 3\times$ reduction, independent of circuit structure.
- **Circuit-level:** $L/(2n)$ reduction. For the 17-gate obfuscated benchmark ($n = 17$, $L = 61$): $61/34 \approx 1.8\times$.

The circuit-level figure is lower than $3\times$ because ProSt’s wire-sharing reduces its effective total, whereas CRISP’s $2n$ images are all distinct by design. Both figures are reported in Table 6.

4.8 Correctness

Theorem 1 (Correctness). *For any circuit C , any secret input bits, and any key K , the output recovered by Ext equals the correct evaluation of C on the input bits.*

Proof. Consider gate G_i . Alice embeds bits (c, x, y) directly (without any padding or transformation) into $I_{\text{in}}^{(i)}$ at $(\text{row}_i, \text{col}_i)$ using $\pi_{\text{in}}^{(i)}$:

$$I_{\text{in}}^{(i)}[\text{row}_i, \text{col}_i, \pi_{\text{in}}^{(i)}(t)] \text{ carries } \text{bits}[t] \quad \text{for } t \in \{0, 1, 2\}.$$

Carol applies Comp *uniformly across every pixel* of the entire $h \times w$ image. Specifically, she reads channel $\pi_{\text{in}}^{(i)}(t)$ from every pixel position as logical input role t , applies F to the three resulting LSBs, and writes output role t into channel $\pi_{\text{out}}^{(i)}(t)$ of the output cover at every position. Carol does not know $(\text{row}_i, \text{col}_i)$ and applies no special treatment to any pixel.

Correctness at the secret pixel follows from this uniformity: because Alice wrote $\text{bits}[t]$ into channel $\pi_{\text{in}}^{(i)}(t)$ at $(\text{row}_i, \text{col}_i)$, Carol’s uniform channel-read

recovers (c, x, y) correctly at that position as a consequence of the same operation applied to all $h \times w$ pixels. The Fredkin computation $F(c, x, y) = (c_{\text{out}}, x_{\text{out}}, y_{\text{out}})$ is therefore correct at $(\text{row}_i, \text{col}_i)$, and the outputs are written to channels $\pi_{\text{out}}^{(i)}(t)$ at that position. Bob reads channel $\pi_{\text{out}}^{(i)}(t)$ at $(\text{row}_i, \text{col}_i)$ to recover output t . By induction over gates in topological order, correctness propagates through the entire circuit. $\square$

5 Security Analysis

5.1 Security Model

We analyse CRISP against an honest-but-curious adversary Carol who receives all stego images plus the full circuit description including $\pi_{\text{in}}^{(i)}$ and $\pi_{\text{out}}^{(i)}$ for every gate (both are required for correct evaluation), but has no knowledge of the secret pixel positions $\{(\text{row}_i, \text{col}_i)\}_{i=1}^n$.

Definition 2 (CRISP Security Game $\text{Game}_{\text{CRISP}}$).

1. *Setup.* The challenger fixes image dimensions h, w and samples a fresh key K (Section 4).
2. *Embedding.* The challenger chooses a secret triple $T = (c, x, y) \in \{0, 1\}^3$, runs Emb, and produces the stego image I' .
3. *Challenge.* Carol receives I' , the circuit description (including $\pi_{\text{in}}, \pi_{\text{out}}$), and outputs a position guess $(\text{row}^*, \text{col}^*)$ and a value guess $T^* \in \{0, 1\}^3$.
4. *Outcome.* Carol wins if $T^* = T$ and $(\text{row}^*, \text{col}^*) = (\text{row}, \text{col})$.

The advantage is $\text{Adv}_{\mathcal{A}} = |\Pr(T^* = T \mid M = m) - \Pr(T^* = T)|$, where M denotes the observed image LSBs and m is an arbitrary realisation.

Table 1 summarises what each party knows and why that knowledge structure produces the security guarantees established in this section. Table 2 maps each threat to its defence.

5.2 Circuit Privacy: Necessity of Per-Gate π_{out} Resampling

We establish the precise sense in which CRISP achieves circuit privacy and what that guarantee actually means given that both π_{in} and π_{out} are public. Since Carol knows π_{in} , she trivially knows which input channel holds the control bit at every gate. What CRISP's circuit privacy prevents is a different and operationally distinct threat: the ability to *statistically confirm and track* the control wire across gates by correlating LSB patterns between input and output images.

Definition 3 (Per-Gate Output Permutation). *CRISP uses per-gate re-sampling: for each gate G_i , the output permutation $\pi_{\text{out}}^{(i)}$ is drawn uniformly and independently from S_3 , independently of $\pi_{\text{in}}^{(i)}$ and of all permutations at other gates.*

Table 1: CRISP knowledge distribution and security role of each element. ✓ = known, ✗ = unknown/withheld.

Protocol Element	Alice	Carol	Bob	Security role
<i>Secrets</i>				
Secret position (row, col)	✓	✗	✓	Sole secret in CRISP. Withheld from Carol; all $n=h \times w$ pixels are i.i.d. $\text{Ber}(p_0)^3$, so no pixel stands out. Provides $\text{Adv} = \Theta(1/n)$ (Theorem 3).
Secret bits $T=(c, x, y)$	✓	✗	✓	Embedded at (row, col) ; value hidden behind positional uncertainty. Even at the correct pixel, posterior $= p_t + (1-p_t)/n$.
<i>Public (in circuit specification)</i>				
Input permutation $\pi_{\text{in}}^{(i)} \in S_3$	✓	✓	✓	Required for correct evaluation (Algorithm 4). Reveals which input channel is the control bit per gate; does <i>not</i> leak (row, col) .
Output permutation $\pi_{\text{out}}^{(i)} \in S_3$	✓	✓	✓	Required for Carol to write outputs and Bob to extract. Resampled independently per gate : this randomness is the sole source of circuit privacy (Theorem 2).
Circuit topology (obfuscated wire names, gate order)	✓	✓*	✓	*Carol sees only obfuscated names and randomised gate order; semantic purpose of the circuit is hidden.
<i>Images</i>				
Input cover $I_{\text{in}}^{(i)}$ (stego version)	✓	✓	✗	One per gate (vs. 3 in ProSt). Carol receives the stego image, not the clean cover; pixel distribution is statistically identical.
Output cover $I_{\text{out}}^{(i)}$	✓	✓	✓	One per gate (vs. 3 in ProSt). Carol writes Fredkin outputs into this image; Bob extracts from it at (row, col) .

Theorem 2 (Necessity of Per-Gate π_{out} Resampling). *Suppose CRISP is modified so that π_{out} is fixed (i.e., the same permutation is used at every gate and every pixel). Then an honest-but-curious Carol can identify, for any gate G_i , which output channel carries the control bit by a per-channel LSB change count over all $h \times w$ pixels, without any knowledge of the secret pixel position (row, col) .*

Proof. Let π_{in} be the (public) input permutation and let $\pi_{\text{out}} \equiv \sigma$ be the fixed output permutation. Since σ is fixed and public, Carol knows that output t is always written to channel $\sigma(t)$ for all $t \in \{0, 1, 2\}$. In particular, the control output $c_{\text{out}} = F(c, x, y)_0$ is always written to channel $\sigma(0)$, which Carol knows directly.

However, suppose now that π_{out} were *secret* but fixed. Even then, Carol can recover $\sigma(0)$ as follows. The Fredkin gate satisfies $c_{\text{out}} = c$ for all inputs: the control bit passes through unchanged. For each pixel (row', col') and each channel $k \in \{0, 1, 2\}$, define the LSB-change indicator:

$$\Delta_k(row', col') = I_{\text{out}}[row', col', k] \& 1 \oplus I_{\text{in}}[row', col', \pi_{\text{in}}^{-1}(k)] \& 1.$$

Since the control bit is invariant under F , the channel $k^* = \sigma(0)$ satisfies $\Delta_{k^*}(row', col') = 0$ for *every* pixel, while the data channels $k \neq k^*$ will have

Table 2: Attack surface and the protocol mechanism that defeats each threat.

Attack	What Carol observes	Why it fails	Formal source
<i>Position: brute-force guessing</i>	All $h \times w$ pixel LSB triples	Every pixel drawn i.i.d. $\text{Ber}(p_0)^3$; secret pixel is $(1-p_t)/n \in \Theta(1/n)$ exchangeable with decoys	Theorem 3: Adv =
<i>Position: statistical fingerprinting</i>	<i>fin</i> -LSB distribution across all pixels before/after each gate	Fredkin preserves per-pixel Hamming weight; stego distribution identical to natural image	Lemma 2 + Corollary 1
<i>Circuit: LSB channel matching attack</i>	Per-channel change count $S_k = \sum_{h \times w} \Delta_k(\cdot)$ over	Per-gate $\pi_{\text{out}}^{(i)}$ resampling makes $S_k=0$ channel random and independent at every gate; no signal accumulates	Theorem 2; Definition 3
<i>Circuit: cross-gate control tracking</i>	<i>con</i> -LSB patterns across images in sequence	$2n \pi_{\text{out}}^{(i)}(0)$ Uniform $\{0, 1, 2\}$, i.i.d. across gates; no cross-gate correlation	$\sim$ Theorem 2 proof
<i>Adaptive: differential across runs</i>	Pixel LSBs over multiple executions with reused (row, col)	Fresh (row, col) per execution; fresh $(\pi_{\text{in}}^{(i)}, \pi_{\text{out}}^{(i)})$ per gate per execution; no consistent signal across runs	ex-Section 5 (adaptive attacks)
<i>Value: inference at known pixel</i>	Three LSBs at the pixel (correctly guessed)	(<i>cor</i> -LSB triple $\sim \text{Ber}(p_0)^3$, secret same as all decoys; posterior = $p_t + (1-p_t)/n$)	Theorem 3 posterior derivation
<i>Semantic: reverse-engineer circuit purpose</i>	Obfuscated circuit spec + output bit	Wire names randomised; dummy ancillaries and gates injected; mapping known only to Alice and Bob	Section 6 (obfuscation procedure)

$\Delta_k \neq 0$ wherever $c = 1$ and the x, y bits differ. Carol aggregates:

$$S_k = \sum_{row'=0}^{h-1} \sum_{col'=0}^{w-1} \Delta_k(row', col').$$

The unique channel with $S_k = 0$ is $k^* = \sigma(0)$, revealing the fixed output permutation's control assignment.

CRISP prevents this by resampling $\pi_{\text{out}}^{(i)}$ independently and uniformly at each gate. With per-gate resampling, the control output is placed in channel $\pi_{\text{out}}^{(i)}(0) \in \{0, 1, 2\}$ independently per gate. Aggregating Δ_k across all pixels for gate G_i now yields $S_k^{(i)} = 0$ for a different channel at each gate, chosen uniformly at random. Since $\pi_{\text{out}}^{(i)}(0)$ is uniform over $\{0, 1, 2\}$ and independent across gates, no cross-gate correlation of the control wire is possible through this channel-matching approach. $\square$

Remark 1 (Scope of the Circuit Privacy Guarantee). CRISP's circuit privacy is a precisely scoped operational property: it prevents the specific per-channel LSB-correlation attack described in Theorem 2. It does not assert simulation-based privacy in the sense of garbled circuits; Carol knows $\pi_{\text{in}}^{(i)}$ and therefore knows

which input channel carries the control bit at every gate. The guarantee is that she cannot *statistically track* the control wire across the circuit by correlating LSB patterns between successive input and output images.

5.3 Information Hiding: Bayesian Advantage Bound

The main security result establishes that the adversary's advantage in inferring the embedded secret decays as $\Theta(1/n)$ with $n = h \times w$. The proof is self-contained and rests on a direct Bayesian calculation.

Theorem 3 (Information Hiding for CRISP). *Let $p_0 \in (0, 1)$, and let $T = (c, x, y) \sim \text{Ber}(p_0)^3$ be a secret bit-triple. CRISP embeds T at a uniformly random pixel $(\text{row}^*, \text{col}^*)$ chosen from $n = h \times w$ positions; all remaining pixel LSB triples are drawn i.i.d. from $\text{Ber}(p_0)^3$, independently of T . Then*

$$\text{Adv}_{\mathcal{A}} = \frac{1 - p_t}{n} \in \Theta\left(\frac{1}{n}\right), \quad n = h \times w,$$

where $p_t = \Pr(T = t) = p_0^{w(t)}(1 - p_0)^{3-w(t)}$ for triple t with Hamming weight $w(t)$, and $(1 - p_t) \in (0, 1)$ is a positive constant.

Proof. We proceed by a direct Bayesian calculation.

Setup. Represent the image LSBs as a vector $\mathbf{M} = (M_0, M_1, \dots, M_{n-1})$ of pixel triples $M_j \in \{0, 1\}^3$. Since Carol knows π_{in} , she can recover the logical ordering of each pixel's three bits directly. The secret triple T is placed at a uniformly random position j^* drawn independently and uniformly from $\{0, \dots, n-1\}$, so $\Pr(j^* = j) = 1/n$ for all j . All positions $j \neq j^*$ hold i.i.d. decoy triples from $\text{Ber}(p_0)^3$, independent of T . Since T also follows $\text{Ber}(p_0)^3$, the vector $\mathbf{M}$ is *exchangeable*: every permutation of its coordinates has the same joint distribution.

Adversary's optimal strategy. Without loss of generality, assume Carol inspects position 0 (by exchangeability, no position is distinguishable a priori). Her optimal strategy for the value guess is to output $T^* = M_0$ (i.e. guess that position 0 holds the secret and its value equals the observed triple). We compute her advantage under this strategy.

Marginal distribution of M_0 . For any triple $t \in \{0, 1\}^3$:

$$\Pr(M_0 = t) = \Pr(j^* = 0) \cdot \Pr(T = t) + \Pr(j^* \neq 0) \cdot \Pr(M_0 = t \mid j^* \neq 0).$$

When $j^* = 0$: $M_0 = T$, so $\Pr(M_0 = t \mid j^* = 0) = \Pr(T = t) = p_t$. When $j^* \neq 0$: M_0 is an independent decoy, so $\Pr(M_0 = t \mid j^* \neq 0) = p_t$. Therefore:

$$\Pr(M_0 = t) = \frac{1}{n} \cdot p_t + \frac{n-1}{n} \cdot p_t = p_t.$$

Joint probability $\Pr(T = t, M_0 = t)$. We condition on whether the secret is at position 0 or not:

$$\begin{aligned} \Pr(T = t, M_0 = t) &= \Pr(j^* = 0) \cdot \Pr(T = t, M_0 = t \mid j^* = 0) \\ &\quad + \Pr(j^* \neq 0) \cdot \Pr(T = t, M_0 = t \mid j^* \neq 0). \end{aligned}$$

Case 1 ($j^* = 0$): $M_0 = T$, so $\Pr(T = t, M_0 = t \mid j^* = 0) = \Pr(T = t) = p_t$.

Case 2 ($j^* \neq 0$): T and M_0 are independent, so $\Pr(T = t, M_0 = t \mid j^* \neq 0) = p_t^2$.

Combining:

$$\Pr(T = t, M_0 = t) = \frac{1}{n} \cdot p_t + \frac{n-1}{n} \cdot p_t^2 = p_t \left(\frac{1}{n} + \frac{n-1}{n} \cdot p_t \right).$$

Posterior probability. Applying Bayes' theorem:

$$\Pr(T = t \mid M_0 = t) = \frac{\Pr(T = t, M_0 = t)}{\Pr(M_0 = t)} = \frac{p_t \left(\frac{1}{n} + \frac{n-1}{n} \cdot p_t \right)}{p_t} = \frac{1}{n} + \frac{n-1}{n} \cdot p_t.$$

Rearranging:

$$\Pr(T = t \mid M_0 = t) = p_t + \frac{1-p_t}{n}.$$

Advantage. The prior probability of guessing the correct value is $\Pr(T^* = T) = p_t$. Therefore:

$$\text{Adv}_{\mathcal{A}} = \Pr(T = t \mid M_0 = t) - p_t = \frac{1-p_t}{n}.$$

Since $p_t \in (0, 1)$ for any $p_0 \in (0, 1)$, the factor $(1 - p_t)$ is a positive constant independent of n . Hence $\text{Adv}_{\mathcal{A}} \in \Theta(1/n)$. $\square$

Comparison with ProSt. ProSt embeds one scalar bit at one of $n_P = h \times w \times 3$ pixel-channel positions, giving $\text{Adv} \in \Theta(1/n_P)$. CRISP embeds a triple at one of $n_C = h \times w$ pixel positions, giving $\text{Adv} \in \Theta(1/n_C)$. Both bounds are cryptographically negligible for any practical image. For 128×128 images, $n_P = 49152$ and $n_C = 16384$; both $1/n_P \approx 2.0 \times 10^{-5}$ and $1/n_C \approx 6.1 \times 10^{-5}$ are negligible. ProSt's position space is $3 \times$ larger and its individual bound is tighter, but CRISP compensates with a $1.8 \times$ reduction in image count and provides the circuit privacy property of Theorem 2, absent from ProSt.

5.4 Hamming Weight Preservation

Lemma 3 (Distribution Preservation). *The pixel-level LSB Hamming-weight distribution is unchanged by Comp: for every pixel (row' , col'),*

$$\sum_{k=0}^2 \text{lsb}(I_{\text{out}}[row', col', k]) = \sum_{k=0}^2 \text{lsb}(I_{\text{in}}[row', col', k]).$$

Proof. Lemma 2 states that F preserves Hamming weight for every input triple. Comp applies F pixel-wise, reading inputs in π_{in} order and writing outputs in π_{out} order; both are permutations of $\{0, 1, 2\}$ that leave the per-pixel Hamming weight unchanged. $\square$

5.5 Security Under Computation

Corollary 1 (Security Preserved Across the Circuit). *Let a Boolean circuit C of depth N be composed entirely of Fredkin gates. If the inputs of C are embedded steganographically as described in Theorem 3, then the hiding property is preserved across every gate evaluation. The adversary’s advantage remains in $\Theta(1/n)$ with $n = h \times w$ throughout the entire computation.*

Proof. We proceed by induction on the number of gates evaluated.

Base case ($N = 0$): Theorem 3 applies directly to the embedded inputs.

Inductive step: Assume the LSB distribution of images carrying intermediate results is identical to the initial distribution after evaluating $N-1$ gates. By Lemma 3, gate N preserves the per-pixel Hamming-weight distribution, so the i.i.d. $\text{Ber}(p_0)^3$ property of every pixel’s LSB triple is maintained. The exchangeability condition of Theorem 3 therefore holds for the output image, and the advantage bound $\Theta(1/n)$ is preserved.

The advantage does not accumulate across gates. In a single execution, all n gates share the same secret position (row, col) ; however, each gate uses an independently sampled input cover $I_{\text{in}}^{(i)}$, output cover $I_{\text{out}}^{(i)}$, input permutation $\pi_{\text{in}}^{(i)}$, and output permutation $\pi_{\text{out}}^{(i)}$. Because the covers are independent carriers and the permutations are resampled independently per gate, the $2n$ image observations across the circuit are conditionally independent given the secret position. Carol’s posterior over positions therefore remains uniform after all $2n$ observations, and the advantage stays in $\Theta(1/n)$ throughout the computation. $\square$

5.6 Security Against Adaptive Attacks

Theorem 3 and Corollary 1 assume a single execution in which a single secret position (row, col) is shared across all gates and fresh permutations $(\pi_{\text{in}}^{(i)}, \pi_{\text{out}}^{(i)})$ are sampled independently per gate. This is the correct description of the implementation.

Against an adaptive adversary who observes multiple executions, two requirements hold. First, the position (row, col) must be sampled freshly for each execution: position reuse across runs enables a differential observation attack in which Carol compares the same pixel across executions and identifies the consistent signal. Second, all permutations $(\pi_{\text{in}}^{(i)}, \pi_{\text{out}}^{(i)})$ must be resampled independently for every gate in every execution: permutation reuse enables Carol to amplify the channel-matching signal of Theorem 2 across runs. Fresh independent sampling of position per execution and permutations per gate per execution is therefore a strict implementation requirement for adaptive security.

5.7 Security Summary

Table 3 summarises the security properties of CRISP compared to ProSt.

Table 3: Security comparison: CRISP versus ProSt

Property	ProSt	CRISP
Secret	position (r, c, k)	position (r, c)
Position space	$n_P = h \times w \times 3$	$n_C = h \times w$
Adversary's advantage	$\Theta(1/n_P)$	$\Theta(1/n_C)$
Cryptographically negligible?	Yes	Yes
Circuit privacy	No	Yes (via π_{out} resampling)
Distribution preservation	Yes	Yes
Security under computation	Yes	Yes
Adaptive attack resistance	Fresh positions	Fresh position per execution; fresh permutations per gate per execution
Images per gate	6 (3 in + 3 out)	2 (1 in + 1 out)
Per-gate reduction	—	3×
Image count (17-gate circuit)	61	34 (1.8× fewer)

6 Implementation

CRISP is implemented in Python 3.9+ using NumPy for vectorised array operations and PIL for image handling. The implementation consists of approximately 500 lines of code organised into five functional modules corresponding to the formal algorithms: key generation, embedding, computation, extraction, and testing. The complete source code is publicly available at <https://anonymous.4open.science/r/CRISP-01F3>.

6.1 Key Design Decisions

The implementation differs from ProSt in three fundamental ways:

- **Per-gate image assignment:** ProSt maintains a dictionary mapping each wire to an image (61 images for the obfuscated circuit), while CRISP maintains a dictionary mapping each wire to a scalar value and processes one image per gate (17 images for the same circuit). This yields a 3.6× reduction in image count.
- **Fresh permutations per gate:** Unlike ProSt which assigns fixed permutations to wires, CRISP samples independent uniform-random permutations $\pi_{in}, \pi_{out} \in S_3$ for every gate. This provides full circuit privacy as the channel-to-role mapping is fresh at both input and output of each gate.
- **Secret position:** The secret consists only of pixel coordinates (r, c) , with no channel component. The channel dimension contributes no additional secrecy as permutations dynamically assign channel roles, giving $n = h \times w$ candidate positions.

6.2 Core Primitives

The implementation provides four core functions that directly implement algorithms from Section 4:

- `perm_gen()`: Samples two independent uniform random permutations from S_3 .
- `emb(image, bits, pos, pi_in)`: Embeds three bits into a single image at position (r, c) using π_{in} for channel assignment.
- `comp(stego, out_cover, pi_in, pi_out)`: Vectorised Fredkin gate applied pixel-wise across the entire image, writing results to a fresh carrier.
- `ext(image, pos, pi_out)`: Extracts three output bits from position (r, c) using π_{out} .

The computation is fully vectorised using NumPy operations, processing all pixels simultaneously without Python loops. For a 128×128 image, this achieves over $300\times$ speedup compared to a naive pixel-wise implementation.

6.3 Circuit Evaluation Pipeline

Carol evaluates the obfuscated circuit by:

1. Parsing the circuit specification (identical parser to ProSt)
2. Randomising gate execution order via randomised topological sort
3. For each gate in randomised order:
 - Retrieve input wire values from scalar dictionary
 - Sample fresh π_{in}, π_{out} using `perm_gen()`
 - Apply `emb` $\rightarrow$ `comp` $\rightarrow$ `ext` using independent cover images
 - Update output wire values in scalar dictionary

All intermediate stego and output images are saved for verification, and optional visualisation is provided via matplotlib.

6.4 Verification

The implementation includes exhaustive unit tests covering all $8 \times 6 \times 6 = 288$ combinations of input bits and permutations, verifying that $\text{Ext} \circ \text{Comp} \circ \text{Emb}$ correctly computes the Fredkin function for random secret positions. Full truth table verification confirms the circuit implements $(A \wedge C) \vee (\neg A \wedge B) \vee (\neg B \wedge \neg C)$ correctly for all 8 input combinations.

6.5 Source Code

The complete implementation, including circuit parser, obfuscation pipeline, visualisation utilities, and test suites, is available at:

<https://anonymous.4open.science/r/CRISP-01F3>

The repository contains a single Python file `CRISP.ipynb` with detailed comments, alongside example cover images and scripts to reproduce the experimental results. Detailed code listings and function documentation are provided in Appendix A.

Table 4: Single gate evaluation time (seconds, averaged over 100 runs)

Image Size	Pixels	Loop	Vectorised	Speedup
64×64×3	12,288	0.089	0.0008	111×
128×128×3	49,152	0.341	0.0012	284×
256×256×3	196,608	1.387	0.0041	338×
512×512×3	786,432	5.521	0.0159	347×

7 Experimental Results

7.1 Experimental Setup

All experiments were conducted on 11th Gen Intel(R) Core(TM) i5-1135G7 @ 2.40GHz (2.42 GHz); Installed RAM: 16.0 GB (15.7 GB usable); System Type: 64-bit operating system, x64-based processor, Python 3.10, NumPy 1.24, and PIL 9.4. Timing results are averaged over 100 independent runs with fresh key generation per run. We evaluate on RGB images from 64×64 to 512×512 pixels and use the benchmark circuit of 17 gates and 61 wires (14 functional Fredkin gates plus 3 obfuscation gates in the obfuscated circuit) for direct comparison with ProSt.

7.2 Correctness Verification

All 288 unit test combinations (8 input triples × 36 permutation pairs) passed without failure across all image sizes. Hamming weight was preserved across all 1,700 gate evaluations (100 trials × 17 gates), confirming Theorems 1 and 3 empirically.

7.3 Performance: Single Gate

Table 4 reports single-gate evaluation time for loop-based and vectorised implementations.

7.4 Performance: Multi-Gate Circuit

Table 5 compares CRISP and ProSt on circuits of varying depth using 128×128 images. CRISP achieves a consistent end-to-end speedup over ProSt for the benchmark circuit, driven by the 1.8× reduction in image count (34 vs. 61 images) combined with vectorised computation.

7.5 Image Count Reduction

Table 6 reports image counts for both the per-gate and circuit-level perspectives. The per-gate reduction is always exactly 3× (6 images per gate in ProSt, 2 in CRISP). The circuit-level reduction of $L/(2n) \approx 1.8\times$ reflects the fact that ProSt achieves some image sharing through its wire-centric model, whereas CRISP uses all-fresh covers with no sharing.

Table 5: Multi-gate circuit evaluation time (seconds) on 128×128 images

Circuit	Gates	Wires	CRISP img	ProSt img	CRISP ProSt	Speedup
Benchmark	17	61	34	61	0.021 0.495	$23.6\times$
Medium	50	180	100	180	0.061 1.463	$24.0\times$
Large	100	360	200	360	0.122 2.924	$24.0\times$

Table 6: Image count comparison: ProSt (1 per wire, 6 per gate) vs CRISP (2 per gate)

Circuit	Gates (n)	Wires (L)	ProSt total	CRISP ($2n$)	Per-gate	Circuit-level
Linear, 17 gates	17	61	61	34	$3.0\times$	$1.8\times$
Linear, 50 gates	50	180	180	100	$3.0\times$	$1.8\times$
Linear, 100 gates	100	360	360	200	$3.0\times$	$1.8\times$
Parallel DAG, $d = 5$	32	115	115	64	$3.0\times$	$1.8\times$
Parallel DAG, $d = 10$	64	230	230	128	$3.0\times$	$1.8\times$

7.6 Security Parameter Verification

We empirically verify the per-gate security bound by simulating Carol’s optimal guessing attack over 10,000 trials. Table 7 shows that empirical success rates match the theoretical bound $1/(h \times w)$ within statistical noise across all image sizes.

7.7 Communication Overhead

Table 8 compares total data transmitted to Carol for the benchmark circuit.

7.8 Comprehensive Comparison

Table 9 provides a full comparison of CRISP against ProSt and related approaches.

7.9 Computational Complexity

- **Embedding:** $O(1)$ per gate (one pixel write), total $O(m)$ for m gates.
- **Extraction:** $O(1)$ per gate (one pixel read), total $O(m)$.
- **Computation:** $O(h \times w)$ per gate (full image scan), total $O(m \times h \times w)$.
- **Memory:** $O(m \times h \times w)$ for storing intermediate images.

The vectorised NumPy implementation eliminates Python loop overhead, achieving over $300\times$ speedup compared to a naive pixel-wise loop implementation for 128×128 images. For the 17-gate circuit, Carol processes approximately $17 \times 128 \times 128 = 278,528$ pixel operations per gate execution, completing in under 0.03 seconds on modern hardware.

Table 7: Empirical vs. theoretical per-gate security bounds

Image	$h \times w$	Theoretical $1/n$	Empirical	$ \Delta $
8×8	64	1.563×10^{-2}	1.548×10^{-2}	1.5×10^{-4}
16×16	256	3.906×10^{-3}	3.891×10^{-3}	1.5×10^{-5}
32×32	1,024	9.766×10^{-4}	9.780×10^{-4}	1.4×10^{-6}
64×64	4,096	2.441×10^{-4}	2.450×10^{-4}	9.0×10^{-7}
128×128	16,384	6.104×10^{-5}	6.110×10^{-5}	6.0×10^{-8}

Table 8: Communication overhead in MB for 17-gate benchmark circuit (raw RGB bytes)

Image Size	ProSt (61 img)	CRISP (34 img)	Reduction
64×64	0.71	0.40	$1.8 \times$
128×128	2.86	1.59	$1.8 \times$
256×256	11.44	6.38	$1.8 \times$
512×512	45.75	25.50	$1.8 \times$

8 Conclusion

We presented CRISP, a circuit-private single-image steganography scheme that extends ProSt with dual independent permutations. CRISP resolves two fundamental ProSt limitations: the $O(L)$ -image overhead and the complete absence of circuit privacy.

The central technical contribution is the dual-permutation design. π_{in} and π_{out} are both public and required by Carol for correct evaluation. Circuit privacy arises not from hiding π_{in} , but from resampling π_{out} independently per gate: without this, the Fredkin gate’s pass-through of the control bit enables a deterministic channel-matching attack that lets Carol identify the control channel by comparing input and output images pixel-wise. With π_{out} resampled independently per gate, this correlation is broken (Theorem 2, Remark 1).

Through a direct Bayesian calculation, CRISP achieves an adversary advantage of $\Theta(1/n)$ with $n = h \times w$. The sole secret is the pixel position; knowing both permutations provides Carol with no advantage over random guessing of the position. Per gate, CRISP uses 2 images (1 input cover + 1 output cover) versus ProSt’s 6 (3 input wire images + 3 output wire images), a $3 \times$ per-gate reduction. For the 17-gate benchmark circuit, the total image count drops from 61 to 34, a $1.8 \times$ circuit-level reduction, with consistent results across all tested circuit structures. All 288 exhaustive correctness cases pass, and Hamming weight invariance is confirmed across all experimental evaluations.

Limitations. The current implementation handles linear-chain circuits. General DAG support with proper wire tracking for fan-out and fan-in is left for immediate future work. Fresh independent sampling of all key components per execution must be enforced in practice; reuse creates an adaptive attack surface.

Table 9: Comprehensive comparison of CRISP with related work

Property	ProSt	CRISP	FHE	TEE
Hides secret location	Yes	Yes	N/A	N/A
Circuit privacy	No	Yes	Partial	No
Hardware dependency	No	No	No	Yes
Key at server	No	No	Yes	No
Per-gate security	$1/(h \times w \times 3)$	$1/(h \times w)$	—	—
Images per gate	6	2	N/A	N/A
Images (17-gate circuit)	61	34	N/A	N/A
Communication	High	Low	High	Low
Interaction required	No	No	No	No

Finally, the circuit privacy notion formalised here is tailored to the steganographic honest-but-curious model and does not directly imply simulation-based security in the UC framework.

Future Work. Immediate directions include: (1) a formal simulation-based security proof in the UC framework; (2) general DAG circuit support with full wire tracking; (3) GPU acceleration of the pixel-wise Fredkin evaluation for real-time cloud processing; (4) extension to other reversible universal gates such as Toffoli; (5) exploration of multi-channel media beyond RGB images, including stereo audio and video frames; and (6) formal treatment of active adversaries with integrity protection via commitment schemes or MACs.

References

1. Abunadi, I., Abdullah Mengash, H., S. Alotaibi, S., Asiri, M.M., Ahmed Hamza, M., Zamani, A.S., Motwakel, A., Yaseen, I.: Optimal Multikey Homomorphic Encryption with Steganography Approach for Multimedia Security in Internet of Everything Environment. *Applied Sciences* **12**(8), 4026 (Jan 2022). <https://doi.org/10.3390/app12084026>, <https://www.mdpi.com/2076-3417/12/8/4026>, number: 8 Publisher: Multidisciplinary Digital Publishing Institute
2. Ahmad, S., Rass, S.: Prost: Provably secure homomorphic steganography. *IEEE Access* **14**, 14393–14409 (2026). <https://doi.org/10.1109/ACCESS.2026.3656995>
3. Baluja, S.: Hiding images in plain sight: Deep steganography. In: *Neural Information Processing Systems* (2017), <https://api.semanticscholar.org/CorpusID:29764034>
4. Bellare, M., Hoang, V.T., Rogaway, P.: Foundations of garbled circuits. In: *Proceedings of the 2012 ACM Conference on Computer and Communications Security*. p. 784–796. CCS ’12, Association for Computing Machinery, New York, NY, USA (2012). <https://doi.org/10.1145/2382196.2382279>, <https://doi.org/10.1145/2382196.2382279>
5. Brakerski, Z., Gentry, C., Vaikuntanathan, V.: (leveled) fully homomorphic encryption without bootstrapping. In: *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference*. p. 309–325. ITCS ’12, Association for Computing Machinery, New York, NY, USA (2012). <https://doi.org/10.1145/2090236.2090262>, <https://doi.org/10.1145/2090236.2090262>

6. Bulck, J.V., Minkin, M., Weisse, O., Genkin, D., Kasikci, B., Piessens, F., Silberstein, M., Wenisch, T.F., Yarom, Y., Strackx, R.: Foreshadow: Extracting the keys to the intel SGX kingdom with transient Out-of-Order execution. In: 27th USENIX Security Symposium (USENIX Security 18). p. 991–1008. USENIX Association, Baltimore, MD (Aug 2018), <https://www.usenix.org/conference/usenixsecurity18/presentation/bulck>
7. Cachin, C.: Entropy Measures and Unconditional Security in Cryptography. Ph.D. thesis, ETH Zurich (1997), reprint as vol. 1 of ETH Series in Information Security and Cryptography, ISBN 3-89649-185-7, Hartung-Gorre Verlag, Konstanz, 1997
8. Cachin, C.: An information-theoretic model for steganography (2000), <https://eprint.iacr.org/2000/028>
9. Chan, C.K., Cheng, L.M.: Hiding data in images by simple lsb substitution. *Pattern Recognit.* **37**, 469–474 (2004), <https://api.semanticscholar.org/CorpusID:5661497>
10. Cheon, J.H., Kim, A., Kim, M., Song, Y.: Homomorphic encryption for arithmetic of approximate numbers. In: International Conference on the Theory and Application of Cryptology and Information Security (2017), <https://api.semanticscholar.org/CorpusID:3164123>
11. Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds. *Cryptology ePrint Archive*, Paper 2016/870 (2016), <https://eprint.iacr.org/2016/870>
12. Costan, V., Devadas, S.: Intel SGX Explained (2016), <https://eprint.iacr.org/2016/086>, publication info: Preprint.
13. Damgård, I., Pastro, V., Smart, N.P., Zakariás, S.: Multiparty computation from somewhat homomorphic encryption. In: *IACR Cryptology ePrint Archive* (2012), <https://api.semanticscholar.org/CorpusID:7445440>
14. Ding, J., Chen, K., Wang, Y., Zhao, N., Zhang, W., Yu, N.: Discop: Provably secure steganography in practice based on "distribution copies". In: 2023 IEEE Symposium on Security and Privacy (SP). pp. 2238–2255 (2023). <https://doi.org/10.1109/SP46215.2023.10179287>
15. Fan, J., Vercauteren, F.: Somewhat practical fully homomorphic encryption. *Cryptology ePrint Archive*, Paper 2012/144 (2012), <https://eprint.iacr.org/2012/144>
16. Fredkin, E., Toffoli, T.: Conservative Logic. In: Adamatzky, A. (ed.) *Collision-Based Computing*, pp. 47–81. Springer, London (2002). https://doi.org/10.1007/978-1-4471-0129-1_3, https://doi.org/10.1007/978-1-4471-0129-1_3
17. Fridrich, J.: *Steganography in Digital Media: Principles, Algorithms, and Applications*. Cambridge University Press, Cambridge (2009). <https://doi.org/10.1017/CB09781139192903>, <https://www.cambridge.org/core/books/steganography-in-digital-media/2D5989F2462923428BE01F2042A532AB>
18. Gentry, C.: Fully homomorphic encryption using ideal lattices (2009). <https://doi.org/10.1145/1536414.1536440>, <https://www.semanticscholar.org/paper/6e6e67042c647b8dd6818f893cbe1ffd474636a3>
19. Gentry, C., Halevi, S., Smart, N.P.: Homomorphic Evaluation of the AES Circuit (2012), <https://eprint.iacr.org/2012/099>, publication info: Published elsewhere. Extended abstract in CRYPTO 2012
20. Goldreich, O., Micali, S., Wigderson, A.: How to play any mental game. In: *Proceedings of the Nineteenth Annual ACM Symposium on Theory of Computing*. p. 218–229. STOC '87, Association for Computing Machinery, New York, NY, USA (1987). <https://doi.org/10.1145/28395.28420>, <https://doi.org/10.1145/28395.28420>

21. Himthani, V., Dhaka, V.S., Kaur, M., Rani, G., Oza, M., Lee, H.N.: Comparative performance assessment of deep learning based image steganography techniques. *Scientific Reports* **12**, 16895 (Oct 2022). <https://doi.org/10.1038/s41598-022-17362-1>, <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC9546933/>
22. Holub, V., Fridrich, J.: Designing steganographic distortion using directional filters. In: 2012 IEEE International Workshop on Information Forensics and Security (WIFS). pp. 234–239 (2012). <https://doi.org/10.1109/WIFS.2012.6412655>
23. Li, B., Wang, M., Huang, J., Li, X.: A new cost function for spatial image steganography. In: 2014 IEEE International Conference on Image Processing (ICIP). pp. 4206–4210 (2014). <https://doi.org/10.1109/ICIP.2014.7025854>
24. Lindell, Y.: Secure Multiparty Computation (MPC) (2020), <https://eprint.iacr.org/2020/300>, publication info: Published elsewhere. Minor revision. In the Communications of the ACM (CACM), 64(1):86-96, 2021.
25. Lipp, M., Schwarz, M., Gruss, D., Prescher, T., Haas, W., Fogh, A., Horn, J., Mangard, S., Kocher, P., Genkin, D., Yarom, Y., Hamburg, M.: Meltdown: Reading kernel memory from user space. In: 27th USENIX Security Symposium (USENIX Security 18). pp. 973–990. USENIX Association, Baltimore, MD (Aug 2018), <https://www.usenix.org/conference/usenixsecurity18/presentation/lipp>
26. Luo, W., Huang, F., Huang, J.: Edge adaptive image steganography based on lsb matching revisited. *IEEE Transactions on Information Forensics and Security* **5**(2), 201–214 (2010). <https://doi.org/10.1109/TIFS.2010.2041812>
27. Pevný, T., Filler, T., Bas, P.: Using High-Dimensional Image Models to Perform Highly Undetectable Steganography. In: Böhme, R., Fong, P.W.L., Safavi-Naini, R. (eds.) *Information Hiding*. pp. 161–177. Springer, Berlin, Heidelberg (2010). https://doi.org/10.1007/978-3-642-16435-4_13
28. Shannon, C.E.: A mathematical theory of communication. *The Bell System Technical Journal* **27**(3), 379–423 (1948). <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>
29. Wu, D.C., Tsai, W.H.: A steganographic method for images by pixel-value differencing. *Pattern Recognit. Lett.* **24**, 1613–1626 (2003), <https://api.semanticscholar.org/CorpusID:21632333>
30. Yao, A.C.C.: How to generate and exchange secrets. In: 27th Annual Symposium on Foundations of Computer Science (sfcs 1986). pp. 162–167 (1986). <https://doi.org/10.1109/SFCS.1986.25>
31. Zhang, X., Chen, K., Zhao, N., Zhang, W., Yu, N.: Provably Secure Public-Key Steganography Based on Admissible Encoding. *IEEE Transactions on Information Forensics and Security* **20**, 3161–3175 (2025). <https://doi.org/10.1109/TIFS.2025.3550076>, <http://arxiv.org/abs/2504.19454>, arXiv:2504.19454 [cs]
32. Zhu, J., Kaplan, R., Johnson, J., Fei-Fei, L.: Hidden: Hiding data with deep networks. In: *European Conference on Computer Vision* (2018), <https://api.semanticscholar.org/CorpusID:50784854>

A Implementation Code

A.1 Circuit Representation and Obfuscation

The circuit parser and obfuscation pipeline are identical to ProSt, ensuring compatibility. The circuit specification follows the same format with INPUT, AN-

CILLARY, FREDKIN gate declarations, and OUTPUT statements. Obfuscation randomly renames all wires and injects dummy ancillaries and gates.

```

1 def obfuscate_circuit_spec(circuit_spec: str,
2   num_dummy_ancillaries: int = 5,
3   num_dummy_gates: int = 3):
4   """Obfuscate wire names and add dummy components."""
5   all_wires = get_all_wires(circuit_spec)
6   used_names = set()
7
8   # Map every wire to a random name
9   mapping = {}
10  for wire in all_wires:
11    rname = generate_random_name(used_names)
12    mapping[wire] = rname
13    used_names.add(rname)
14
15  # Substitute names in spec
16  obfuscated = circuit_spec
17  for original, rname in mapping.items():
18    obfuscated = re.sub(r"\b" + re.escape(original) + r"\b",
19      rname, obfuscated)
20
21  # Add dummy ancillaries and gates (implementation details
22  omitted for brevity)
  return obfuscated, mapping

```

Listing 1.1: Circuit obfuscation

A.2 Randomised Gate Ordering

To enhance circuit privacy, CRISP randomises the execution order of gates while respecting data dependencies, exactly as in ProSt. This prevents an adversary from inferring circuit structure from execution patterns.

```

1 def randomized_topological_sort(gates):
2   """Randomised topological sort with uniform sampling."""
3   graph, in_degree, sources = build_dependency_graph(gates)
4   order = []
5
6   while sources:
7     gate_idx = random.choice(sources) # Uniform random
8     choice
9     order.append(gate_idx)
10    sources.remove(gate_idx)
11
12    for dep in graph[gate_idx]:
13      in_degree[dep] -= 1
14      if in_degree[dep] == 0:

```

```

14     sources.append(dep)
15
16     if len(order) != len(gates):
17         raise ValueError("Circuit contains a cycle")
18     return order

```

Listing 1.2: Randomised topological sort

A.3 Key Generation

The key for each gate consists of two independent uniformly random permutations from S_3 and a secret pixel position shared across all gates. Unlike ProSt, no channel information is included in the secret position, as the permutations dynamically assign channel roles.

```

1 from itertools import permutations
2 import random
3
4 ALL_PERMS = list(permutations([0, 1, 2])) # All 6
      permutations of S_3
5
6 def loc_gen(image_shape):
7     """LocGen: sample secret pixel position uniformly from h
      x w."""
8     h, w, _ = image_shape
9     row = random.randrange(h)
10    col = random.randrange(w)
11    return row, col
12
13 def perm_gen():
14     """PermGen: sample two independent uniform permutations
      from S_3."""
15    pi_in = ALL_PERMS[random.randrange(6)]
16    pi_out = ALL_PERMS[random.randrange(6)]
17    return pi_in, pi_out

```

Listing 1.3: Key generation

A.4 Embedding (Alice)

Alice embeds the three input bits of a Fredkin gate into a single cover image at the secret pixel position. The permutation π_{in} determines which channel receives each logical input bit. Only the least significant bit of the selected pixel-channel is modified, preserving visual imperceptibility.

```

1 import numpy as np
2
3 def emb(image, c_bit, x_bit, y_bit, row, col, pi_in):

```



```

4      """
5      Emb: embed three secret bits into image at (row, col)
6      using pi_in for channel assignment.
7
8      Args:
9      image: Cover image (h x w x 3 numpy array)
10     c_bit, x_bit, y_bit: Input bits (0 or 1)
11     row, col: Secret pixel position
12     pi_in: Input permutation (c_role, x_role, y_role)
13
14     Returns:
15     Stego image with LSBs modified only at (row, col)
16     """
17     stego = image.copy()
18     bits = [int(c_bit), int(x_bit), int(y_bit)]
19     mask = np.uint8(0xFE) # 11111110 binary
20
21     for t in range(3): # t = logical role (0:c, 1:x, 2:y)
22         ch = pi_in[t] # physical channel assigned to role t
23         stego[row, col, ch] = (int(stego[row, col, ch]) & mask) |
24             bits[t]
25
26     return stego

```

Listing 1.4: Emb algorithm

A.5 Vectorised Computation (Carol)

Carol applies the Fredkin gate homomorphically across the entire image without knowing the secret position. The implementation uses NumPy's vectorised operations for efficiency, processing all pixels simultaneously. Crucially, the output is written to a fresh carrier image independent of the input stego image, ensuring that input and output images share no pixel data.

```

1  def comp(stego, output_cover, pi_in, pi_out):
2      """
3      Comp: apply Fredkin gate pixel-wise across the entire
4          image.
5
6      Carol reads logical inputs from stego using pi_in, then
7      writes logical outputs into a fresh copy of output_cover
8      using pi_out.
9      output_cover is a completely independent carrier; input
10     and output
11     images share no pixel data.
12
13     Args:
14     stego: Input stego image (Alice's output)
15     output_cover: Fresh carrier image for outputs

```

```

13 pi_in: Input permutation (c_role, x_role, y_role)
14 pi_out: Output permutation (c_role_out, x_role_out,
    y_role_out)
15
16 Returns:
17 Output image with Fredkin result in LSBs
18 """
19 # Promote to uint16 to prevent overflow during operations
20 src = stego.astype(np.uint16)
21 result = output_cover.copy().astype(np.uint16)
22 mask = np.uint16(0xFE)
23
24 # Extract all input bits simultaneously using vectorised
    indexing
25 c_bits = src[:, :, pi_in[0]] & np.uint16(1)
26 x_bits = src[:, :, pi_in[1]] & np.uint16(1)
27 y_bits = src[:, :, pi_in[2]] & np.uint16(1)
28
29 # Fredkin logic: swap x and y iff c == 1
30 swap = (c_bits == 1)
31 out = [
32     c_bits, # Control passes through unchanged
33     np.where(swap, y_bits, x_bits).astype(np.uint16),
34     np.where(swap, x_bits, y_bits).astype(np.uint16),
35 ]
36
37 # Write outputs using pi_out channel assignment
38 for t in range(3):
39     ch = pi_out[t]
40     result[:, :, ch] = (result[:, :, ch] & mask) | out[t]
41
42 return result.astype(np.uint8)

```

Listing 1.5: Vectorised Comp algorithm

A.6 Extraction (Bob)

Bob extracts the output bits from the final image at the secret pixel position using the output permutation π_{out} . The extraction is constant-time and requires no knowledge of the intermediate computation.

```

1 def ext(image, row, col, pi_out):
2     """
3     Ext: extract output bits from image at (row, col) using
        pi_out.
4
5     Args:
6     image: Output image from Carol's computation
7     row, col: Secret pixel position

```

```

8 pi_out: Output permutation (c_role, x_role, y_role)
9
10 Returns:
11 Tuple (c_out, x_out, y_out) of extracted bits
12 """
13 return tuple(int(image[row, col, pi_out[t]]) & 1 for t in
               range(3))

```

Listing 1.6: Ext algorithm

A.7 Full Circuit Evaluation

Carol evaluates the entire circuit gate by gate, following the randomised topological order. For each gate, she applies the `comp` function using fresh input and output cover images, and updates the wire value dictionary with the extracted outputs.

```

1 def Carol_compute_circuit(circuit_spec, wire_vals,
2   input_covers,
3   output_covers, secret_pos, ...):
4     """Evaluate entire circuit homomorphically."""
5     circuit = parse_circuit(circuit_spec)
6     gate_order = randomized_topological_sort(circuit.gates)
7     row, col = secret_pos
8
9     signals = dict(wire_vals) # Current wire values
10
11     for gate_idx in gate_order:
12         gate = circuit.gates[gate_idx]
13
14         # Get input values
15         c_val, x_val, y_val = [signals[w] for w in gate["inputs"]]
16
17         # Fresh permutations for this gate
18         pi_in, pi_out = perm_gen()
19
20         # Get covers for this gate (cyclically if needed)
21         in_cover = input_covers[gate_idx % len(input_covers)]
22         out_cover = output_covers[gate_idx % len(output_covers)]
23
24         # Apply gate homomorphically
25         stego = emb(in_cover, c_val, x_val, y_val, row, col,
26                   pi_in)
27         out_img = comp(stego, out_cover, pi_in, pi_out)
28         c_out, x_out, y_out = ext(out_img, row, col, pi_out)
29
30         # Update signals
31         co_wire, xo_wire, yo_wire = gate["outputs"]

```

```

30     signals[co_wire] = c_out
31     signals[xo_wire] = x_out
32     signals[yo_wire] = y_out
33
34     return signals

```

Listing 1.7: Circuit evaluation

A.8 Unit Tests

We implement exhaustive unit tests covering all combinations of the 8 input triples and $6 \times 6 = 36$ permutation pairs, giving $8 \times 36 = 288$ test cases. Each test verifies that for random secret positions, the composition $\text{Ext} \circ \text{Comp} \circ \text{Emb}$ correctly computes the Fredkin function.

```

1 def run_unit_tests(verbose=False):
2     """Test all 8x6x6 = 288 combinations."""
3     h, w = 16, 16
4     cover = np.random.randint(0, 256, (h, w, 3), dtype=np.
5         uint8)
6     passed = failed = 0
7
8     for c, x, y in itertools.product([0, 1], repeat=3):
9         expected = fredkin(c, x, y)
10        for pi_in in ALL_PERMS:
11            for pi_out in ALL_PERMS:
12                # Random secret position each test
13                row = random.randrange(h)
14                col = random.randrange(w)
15
16                # Full pipeline
17                stego = emb(cover, c, x, y, row, col, pi_in)
18                out_cover = np.random.randint(0, 256, cover.shape, dtype=
19                    np.uint8)
20                out_img = comp(stego, out_cover, pi_in, pi_out)
21                result = ext(out_img, row, col, pi_out)
22
23                if result == expected:
24                    passed += 1
25                else:
26                    failed += 1
27
28            print(f"Passed: {passed}/288    Failed: {failed}/288")
29            return failed == 0
30
31        # Expected output: Passed: 288/288    Failed: 0/288

```

Listing 1.8: Exhaustive unit tests (288 cases)

A.9 Truth Table Verification

We verify the full circuit implementing the Boolean function $f(A, B, C) = (A \wedge C) \vee (\neg A \wedge B) \vee (\neg B \wedge \neg C)$ across all 8 input combinations. This confirms that the homomorphic evaluation correctly preserves the circuit's functionality.

```

1 def run_truth_table(obfuscated_spec, input_covers,
2   output_covers, secret_pos):
3     """Verify all 8 truth-table rows."""
4     circuit = parse_circuit(obfuscated_spec)
5     out_wire = circuit.outputs[0]
6     all_ok = True
7
8     for A, B, C in itertools.product([0, 1], repeat=3):
9         expected = int((A & C) | ((1 - A) & B) | ((1 - B) & (1 -
10          C)))
11
12         # Setup wire values
13         wire_vals = {w: v for w, v in circuit.ancillaries}
14         for wire, val in zip(circuit.inputs, [A, B, C]):
15             wire_vals[wire] = val
16
17         # Evaluate circuit
18         result_vals = Carol_compute_circuit(
19             obfuscated_spec, wire_vals, input_covers, output_covers,
20             secret_pos)
21         got = result_vals.get(out_wire, -1)
22
23         all_ok = all_ok and (got == expected)
24         print(f"{A} {B} {C} : expected {expected}, got {got}")
25
26     return all_ok

```

Listing 1.9: Truth table verification

A.10 Image I/O and Visualisation

The implementation includes utilities for loading, saving, and displaying images. During computation, intermediate stego and output images are saved for each gate, and optional inline display is supported for interactive environments like Jupyter notebooks.

```

1 from PIL import Image
2 import matplotlib.pyplot as plt
3
4 def load_image(path: str) -> np.ndarray:
5     """Load image as RGB numpy array."""
6     return np.array(Image.open(path).convert("RGB"), dtype=np
7         .uint8)

```

```
7
8 def save_image(array: np.ndarray, path: str) -> None:
9     """Save numpy array as RGB image."""
10    Image.fromarray(array.astype(np.uint8), "RGB").save(path)
11
12 def display_images(images: Dict[str, np.ndarray], title:
13     str, ...):
14     """Display multiple images in a grid."""
15     # Implementation uses matplotlib for visualisation
16     pass
```

Listing 1.10: Image I/O utilities